

VIT BHOPAL UNIVERSITY

PROJECT REPORT

CIFAR-10 IMAGE CLASSIFICATION USING ALL-CNN

Submitted By	Zainab Hasan
Registration No.	25MIB10050
Programme	Integrated M.Tech in AI and Bioinformatics
Course	Fundamentals of AI and ML
Academic Year	2026–27

A deep learning project for image classification on the CIFAR-10 dataset.

TABLE OF CONTENTS

1. Introduction
2. Problem Statement
3. Objectives
4. Functional Requirements
5. Non-Functional Requirements
6. System Architecture
7. Design Diagrams
8. Design Decisions & Rationale
9. Implementation Details
10. Dataset Description
11. Screenshots / Results
12. Testing Approach
13. Challenges Faced
14. Learnings & Key Takeaways
15. Future Enhancements
16. References

1. INTRODUCTION

Image classification is a fundamental computer vision task in which a machine learning model assigns an input image to one of a predefined set of categories. The CIFAR-10 dataset provides a standard environment for studying this task because it contains small colour images belonging to ten object classes. The supplied project uses a convolutional neural network approach to learn visual patterns from these images and classify previously unseen examples.

The project is centred on an All-CNN architecture. The notebook defines a sequence of convolutional layers, dropout layers, and global average pooling followed by softmax classification. The preprocessing stage converts image pixel values to the 0–1 range and converts integer class labels into ten-dimensional one-hot vectors.

The overall workflow consists of dataset loading, visualization, preprocessing, model construction, training, inference, visual verification, and saving the trained model. The supplied project materials describe the objective as building a neural network capable of classifying CIFAR-10 images and learning how image data can be prepared and analysed for deep learning.

The project is relevant to Fundamentals of AI and ML because it demonstrates supervised multi-class classification, neural network architecture, data preprocessing, optimization, model training, and prediction.

2. PROBLEM STATEMENT

The problem addressed by this project is the automatic classification of low-resolution colour images into one of ten predefined CIFAR-10 categories. Manual classification is not scalable for large collections of images, while a trained neural network can learn discriminative visual features directly from labelled examples.

Problem statement: Develop a deep learning-based image classification system that accepts CIFAR-10 images as input and predicts the corresponding object category from ten classes: airplane, automobile, bird, cat, deer, dog, frog, horse, ship, and truck.

The project scope is limited to the CIFAR-10 dataset and the classification workflow implemented in the supplied notebook. It does not attempt object detection, image segmentation, or classification outside the ten CIFAR-10 classes.

3. OBJECTIVES

- Build a neural network capable of classifying CIFAR-10 images.
- Understand the structure and characteristics of the CIFAR-10 dataset.
- Prepare image and label data in a form suitable for Keras-based neural network training.
- Normalize image pixel values to improve the numerical representation used during training.
- Convert integer class labels into one-hot encoded vectors.
- Implement and train an All-CNN-style architecture
- Evaluate the model on previously unseen test images.
- Visually compare predicted and actual labels on a sample of test images

- Save the trained model for later use.

4. FUNCTIONAL REQUIREMENTS

The project contains the following major functional modules. These satisfy the requirement for at least three major functional modules with a clear input/output workflow.

4.1 Dataset Loading & Visualization

- Input: CIFAR-10 dataset accessed through the Keras dataset utility.
- Processing: Load training and testing images and inspect sample images.
- Output: Training/test arrays and visual sample grids.

4.2 Data Preprocessing

- Input: Raw CIFAR-10 images and integer class labels.
- Processing: Convert images to float32, normalize pixel values by 255, and one-hot encode labels.
- Output: Normalized image arrays and ten-class encoded target vectors.

4.3 Model Construction & Training

- Input: Preprocessed training data.
- Processing: Build the convolutional network, compile it with categorical cross-entropy and SGD, and train using mini-batches.
- Output: Trained model and training history.

4.4 Model Inference & Visual Verification

- Input: A batch of nine test images.
- Processing: Generate class probabilities and select the highest-probability class.
- Output: Predicted class indices and a 3×3 visual comparison of predicted versus actual labels.

4.5 Model Persistence

- Input: Trained Keras model.
- Output: Saved HDF5 model file named `cifar10_classifier_model.h5`.

5. NON-FUNCTIONAL REQUIREMENTS

- Performance: The system should complete preprocessing and inference efficiently enough for CIFAR-10-sized inputs. Training is computationally intensive because the network contains a large number of parameters.
- Usability: The notebook presents the workflow in sequential stages and displays intermediate outputs, images, predictions, and model information.
- Reliability: A fixed NumPy seed is used for reproducibility of the numerical setup, and the model is saved after execution.
- Maintainability: The notebook separates the workflow into recognizable stages such as data loading, preprocessing, model construction, training, and inference.

- **Resource Efficiency:** The project acknowledges the significant computational cost of deep CNN training and discusses GPU acceleration as a practical solution.
- **Validation / Error Awareness:** The inference stage checks prediction output structure and demonstrates that each prediction probability vector sums to 1.

The project guidelines require at least four non-functional requirements; the report specifies six relevant requirements for this implementation.

6. SYSTEM ARCHITECTURE

The system follows a sequential deep-learning workflow. Data is first loaded from the CIFAR-10 dataset and inspected. The images and labels are then transformed into suitable numerical representations. The neural network receives normalized 32×32 RGB images and produces a ten-class probability distribution. The highest-probability class is selected as the prediction.

Stage	Function
CIFAR-10 Dataset	Provides labelled training and testing images.
Data Loading	Loads training and test arrays through Keras.
Visualization	Displays sample images for inspection.
Preprocessing	Normalizes pixels and encodes labels.
All-CNN Model	Extracts visual features and produces class probabilities.
Training / Inference	Optimizes the model and generates predictions.
Class Prediction	Selects the highest-probability class.
Model Saving	Stores the trained model for later use.

7. DESIGN DIAGRAMS

7.1 Workflow Diagram

Step	Process
1	START
2	Load CIFAR-10
3	Inspect sample images
4	Normalize pixels
5	One-hot encode labels
6	Construct CNN
7	Compile model
8	Train model
9	Predict test batch
10	Compare predicted vs actual
11	Save model
12	END

7.2 Use Case Diagram

The primary actor is the project user/developer operating the notebook. The main use cases are loading the dataset, visualizing samples, preprocessing data, building the model, training the model, generating predictions, visualizing predictions, and saving the model.

Actor	Use Case	Output
User / Developer	Load dataset	Training and testing arrays
User / Developer	Preprocess data	Normalized images and encoded labels
User / Developer	Train model	Trained neural network
User / Developer	Run inference	Class probabilities / prediction
User / Developer	Visualize predictions	Prediction grid
User / Developer	Save model	HDF5 model file

7.3 Sequence Diagram

Actor / Component	Action
User / Developer	Runs notebook workflow
Notebook Controller	Requests dataset
CIFAR-10 Dataset Utility	Returns training and test arrays
Preprocessing	Normalizes images and encodes labels
All-CNN Model	Receives data and performs training/inference
Prediction / Visualization	Displays predicted and actual labels

7.4 Component Diagram

Component	Responsibility
Data Loading	Loads CIFAR-10 training and test arrays.
Visualization	Displays sample images and prediction grids.
Preprocessing	Normalizes pixels and encodes class labels.
All-CNN Architecture	Defines convolutional, dropout, stride-based reduction, global-average-pooling and softmax layers.
Training	Compiles and fits the model using SGD and categorical cross-entropy.
Inference	Generates class probabilities and predicted class indices.
Persistence	Saves the Keras model as an HDF5 file.

7.5 Storage / ER Diagram

A relational database is not used in the supplied project. The principal persistent artifact is the trained HDF5 model file. Therefore, an ER diagram is not applicable to the current implementation. The project instead uses in-memory NumPy arrays and a saved model file.

8. DESIGN DECISIONS & RATIONALE

8.1 Selection of CIFAR-10

CIFAR-10 is used because it provides labelled colour images across ten object categories and is directly accessible through the Keras dataset utility used in the notebook. The supplied report identifies 60,000 images in total, with 50,000 training images and 10,000 testing images.

8.2 Selection of Convolutional Neural Networks

CNN-based architectures are appropriate for image classification because they learn spatial patterns through convolutional filters. The project specifically defines an All-CNN-style network rather than using fully connected layers as the main feature extractor.

8.3 All-CNN Architecture

The notebook's architecture uses 3×3 convolutional layers with 96 and 192 filters, stride-based spatial reduction, dropout, 1×1 convolutional layers, global average pooling, and a final softmax activation. The architecture description in the notebook lists the sequence from the 32×32 RGB input through the convolutional and pooling stages to ten-way softmax classification.

8.4 Preprocessing Choices

Pixel values are converted to float32 and divided by 255 so that image intensities fall in the 0–1 range. Labels are transformed from integer class IDs to ten-element one-hot vectors, which matches the categorical cross-entropy training setup.

8.5 Optimization

The notebook specifies SGD with a learning rate of 0.01, momentum of 0.9, and Nesterov acceleration. The model is compiled with categorical cross-entropy loss and accuracy as the tracked metric.

8.6 Reproducibility

A NumPy seed value of 6 is set during preprocessing. The training uses a batch size of 32 and a nominal ten epochs in the supplied training cell.

9. IMPLEMENTATION DETAILS

9.1 Data Loading

The notebook uses `tensorflow.keras.datasets.cifar10.load_data()`. The recorded shapes are (50,000, 32, 32, 3) for training images and (10,000, 32, 32, 3) for test images. Each image is therefore a 32×32 RGB image.

9.2 Data Normalization

The image arrays are converted to float32 and divided by 255.0. This converts the original 8-bit pixel representation to normalized floating-point values.

9.3 Label Encoding

The integer labels are converted with `to_categorical(..., 10)`, producing arrays with shape (50,000, 10) for the training labels and ten output classes.

9.4 Model Definition

The notebook defines an `allcnn(weights=None)` function. It constructs the convolutional network using Keras Sequential layers. The final layer sequence applies global average pooling followed by softmax, producing ten class probabilities.

9.5 Training

The model is compiled using categorical cross-entropy and SGD. The supplied training cell specifies a batch size of 32, ten epochs, validation data from the CIFAR-10 test set, and shuffling.

9.6 Inference

A batch containing nine test images is selected. The model produces a ten-element probability vector for each image. `argmax` is then used to obtain the predicted class index, and the results are plotted against the actual CIFAR-10 labels.

9.7 Model Saving

The notebook saves the model to `cifar10_classifier_model.h5`. The supplied notebook records a Keras warning that HDF5 is a legacy model format and recommends the native Keras format for new projects.

10. DATASET DESCRIPTION

The project uses the CIFAR-10 dataset accessed through TensorFlow/Keras. The supplied project report describes it as a collection of 60,000 low-resolution colour images distributed across ten categories: airplane, automobile, bird, cat, deer, dog, frog, horse, ship, and truck. It further records 50,000 images for training and 10,000 for testing.

Property	Value
Dataset	CIFAR-10
Total images	60,000
Training images	50,000
Testing images	10,000
Image dimensions	$32 \times 32 \times 3$
Number of classes	10
Output representation	10-class one-hot vector / softmax probabilities

Class labels: 0 – airplane; 1 – automobile; 2 – bird; 3 – cat; 4 – deer; 5 – dog; 6 – frog; 7 – horse; 8 – ship; 9 – truck.

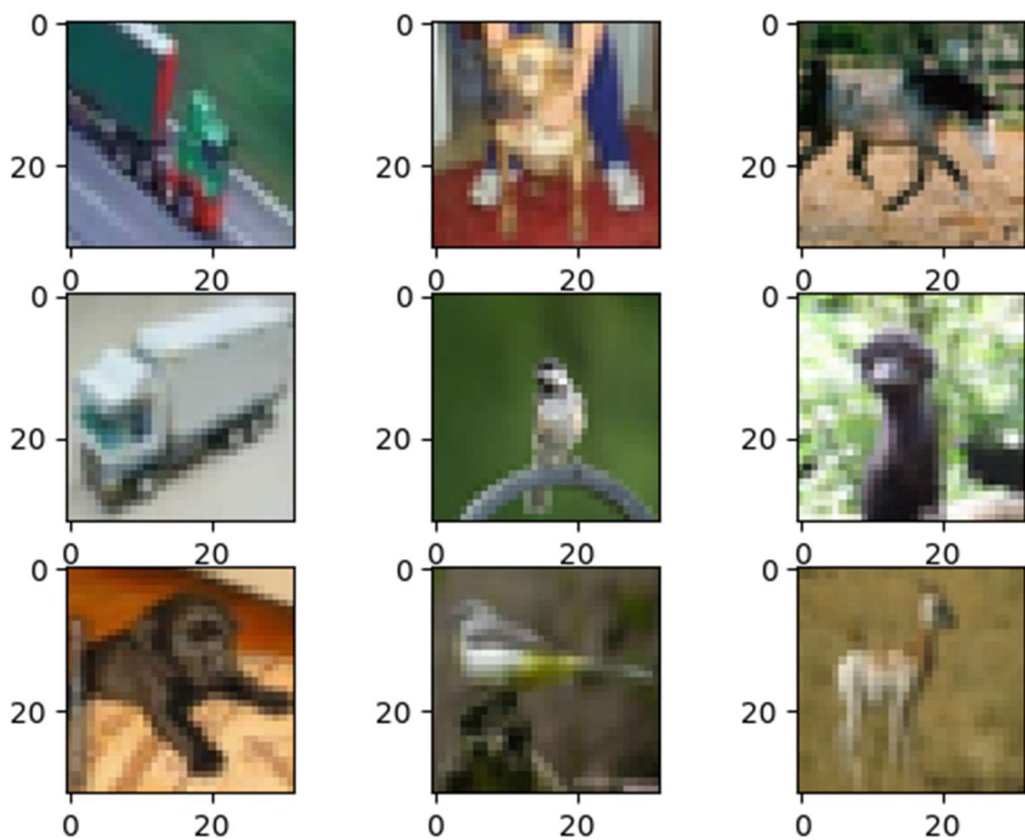


Figure: Sample CIFAR-10 training images generated in the supplied notebook.

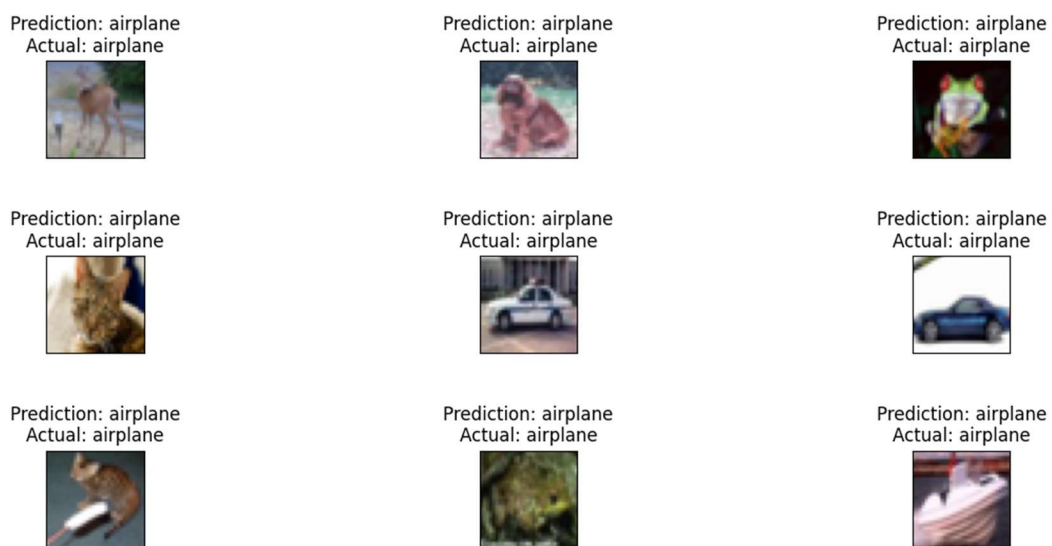


Figure: Prediction visualization output embedded in the supplied notebook.

11. SCREENSHOTS / RESULTS

The supplied project materials report an outcome of approximately 90% accuracy on unseen test images. The original report states that the model classified 90% of previously unseen images, while the README also records approximately 90% test accuracy.

The notebook also contains a visual verification stage in which nine test images are passed to the model and displayed with predicted and actual class labels.

Important reproducibility note: The supplied notebook contains an All-CNN model definition, but a later code cell reassigns model to an untrained MobileNetV2 instance with weights=None. The supplied notebook's recorded ten-epoch training output also stops during epoch 9, and the later inference output shown in the notebook consists of uniform 0.1 probabilities. Therefore, the approximately 90% figure above is presented as the reported result in the supplied project materials, not as an independently verified result from the supplied notebook state. This implementation inconsistency should be resolved before treating the accuracy as reproducible.

The supplied training log records increasing training accuracy through epoch 8, reaching 0.5736, while validation accuracy reaches 0.5342 at epoch 8. The run then begins epoch 9 but does not contain a completed epoch-9 result in the notebook output.

12. TESTING APPROACH

Testing in the supplied project is primarily based on validation through model training output and qualitative inference checks.

12.1 Data Shape Validation

- Training image array is recorded as (50,000, 32, 32, 3).
- Test image array is recorded as (10,000, 32, 32, 3).
- Encoded training labels are recorded as (50,000, 10).
- The test inference batch contains nine images.

12.2 Prediction Output Validation

- Each prediction contains ten class probabilities.
- The notebook explicitly checks the sum of each prediction vector; each displayed sum is 1.0.
- The predicted class is obtained using argmax over the ten output probabilities.

12.3 Visual Testing

A 3×3 grid is generated to compare each predicted label with its actual CIFAR-10 label. This provides a qualitative check of inference behaviour.

12.4 Training Monitoring

Training and validation accuracy/loss are printed during model fitting. The supplied run shows training accuracy increasing from 0.2727 in epoch 1 to 0.5736 in epoch 8, while validation accuracy varies and reaches 0.5342 in epoch 8.

13. CHALLENGES FACED

- Training the convolutional model on a CPU is computationally expensive. The supplied notebook records very long epoch times, with the first epoch taking approximately 3983 seconds.
- The notebook explains that GPU acceleration can substantially reduce the training time of deep neural networks.
- The supplied training run did not complete all ten recorded epochs; the output begins epoch 9 without a completed result.
- The notebook contains a model-definition inconsistency: an All-CNN function is defined, but a later cell creates a MobileNetV2 model with no pretrained weights.
- The final inference output in the supplied notebook gives equal 0.1 probabilities across all ten classes, which indicates that the later inference model state does not correspond to the reported approximately 90% result.
- The HDF5 model-save warning indicates that the format used is considered legacy by current Keras and that the native Keras format is recommended for new work.

14. LEARNINGS & KEY TAKEAWAYS

- Image data must be transformed into a numerical representation suitable for neural network training.
- Pixel normalization is a basic but important preprocessing step for image models.
- One-hot encoding aligns integer class labels with multi-class neural network outputs.
- CNNs can learn spatial features through stacked convolutional layers.
- Dropout can be incorporated into a CNN architecture as a regularization mechanism.
- Global average pooling can connect convolutional feature maps to class predictions without relying on a large fully connected feature-extraction stage.
- Optimization settings such as learning rate, momentum, batch size, and epoch count affect training behaviour.
- Training time is an important practical consideration in deep learning, particularly for CPU-based execution.
- Model inference should be checked both quantitatively and visually.
- Reproducibility requires consistency between the architecture trained, the weights loaded, the model used for inference, and the reported results.

15. FUTURE ENHANCEMENTS

- Resolve the All-CNN/MobileNetV2 model-state inconsistency and produce a clean, reproducible training and inference pipeline.
- Use the native Keras model format instead of HDF5 for new model exports.
- Add data augmentation to increase variation in the training images and help address overfitting.
- Experiment with modern architectures such as ResNet or DenseNet, as suggested in the supplied project report.
- Perform systematic hyperparameter tuning for learning rate, batch size, dropout rate, optimizer and number of epochs.

- Add quantitative metrics such as a classification report and confusion matrix.
- Add a dedicated script or interface for classifying user-provided CIFAR-10-sized images.
- Compare the All-CNN architecture with other CNN architectures under the same training and evaluation conditions.
- Investigate GPU-based training to reduce execution time.

16. REFERENCES

1. VITyarthi – Build Your Own Project: General Project Instructions & Submission Guidelines. Supplied course instruction document.
2. A. Krizhevsky, “Learning Multiple Layers of Features from Tiny Images,” University of Toronto, 2009.
3. J. T. Springenberg, A. Dosovitskiy, T. Brox, and M. Riedmiller, “Striving for Simplicity: The All Convolutional Net,” arXiv:1412.6806, 2014.
4. TensorFlow, “tf.keras.datasets.cifar10.load_data,” TensorFlow API Documentation.
5. Keras, “CIFAR10 small images classification dataset,” Keras 3 API Documentation.
6. Keras, “CutMix data augmentation for image classification,” Keras Examples, Computer Vision.